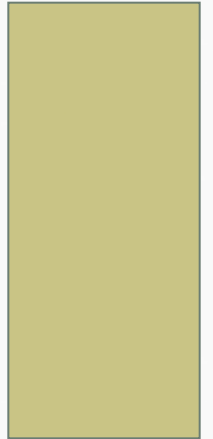


DATA STRUCTURES

LECTURE 13



INTRODUCTION TO THE STANDARD TEMPLATE LIBRARY (STL)

- Software reuse. 
- Data structures and algorithms commonly used by C++ programmers
- Part of C++ Standard Library.

KEY COMPONENTS OF STL

1. Containers ✓

- A way of storing data (e.g. stack, linked list)
- popular data structures in the form of template classes

2. Algorithms ✓

- Procedures that are applied to containers to process their data in various ways
- E.g. sort, copy, search
- Represented as (stand-alone) template functions

KEY COMPONENTS OF STL

3. Iterators ✓

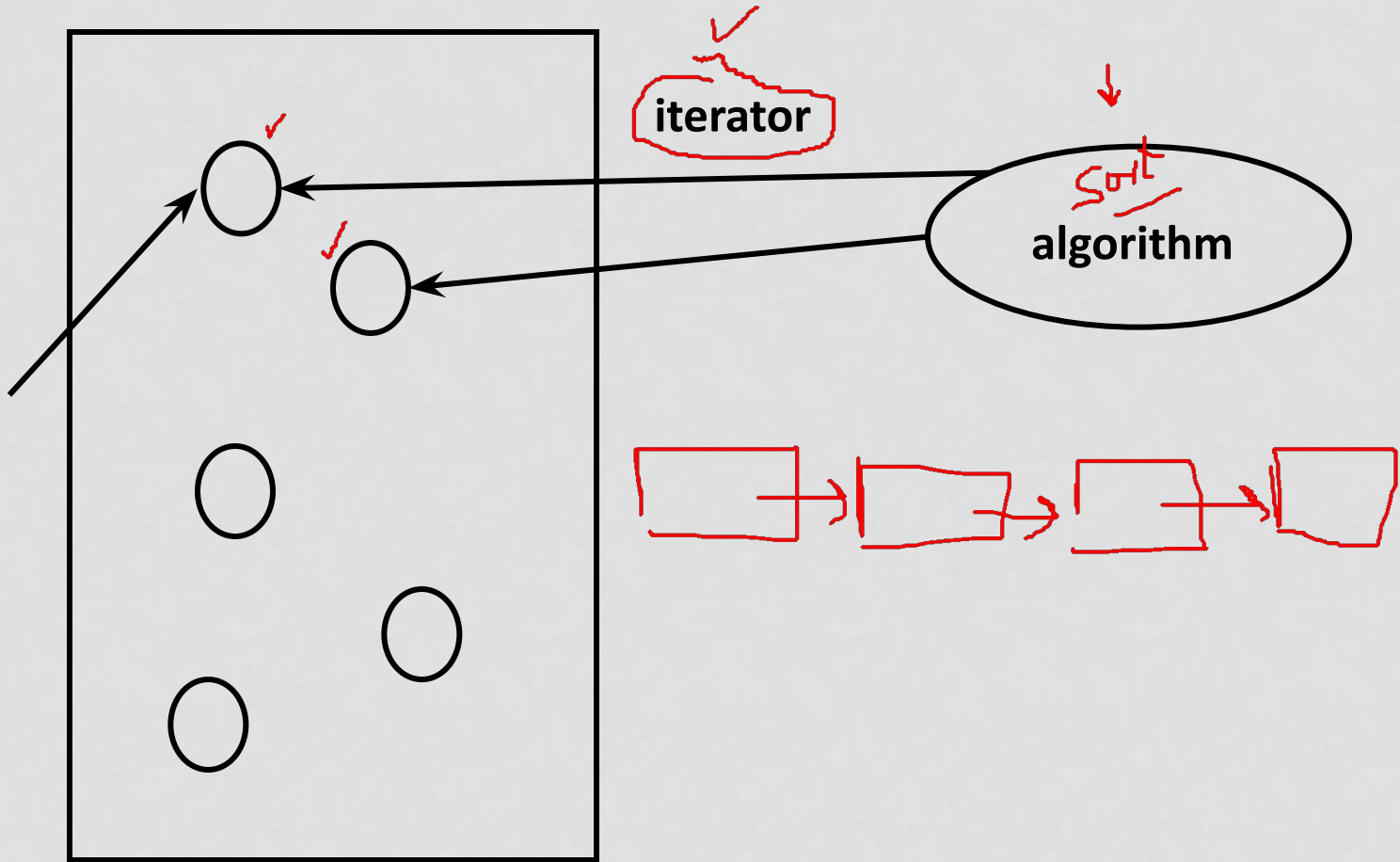
- Generalization of the pointer concept
- Point to elements in a container
- Connect algorithm with container



KEY COMPONENTS OF STL

container

Element
(could be
a
primitive
data type
or an
object)



STL CONTAINERS

STL COMPONENT 1

STL CONTAINERS

- A way to store data. Data can be any type:
 - Built – in type
 - Class objects
- 7 basic kinds of containers (+ 3 derived)
- You can create your own also (by inheritance)

STL CONTAINER CATEGORIES

1. ✓ Sequence containers (linear data structures)
 - ✓ Vector (relaxed array, can change size dynamically)
 - ✓ List (easy insertion/deletion)
 - ✓ Deque (double ended queue, insert/remove at both ends)
2. ✓ Associative containers (non-linear data structures)
 - ✓ Set (key/value)
 - ✓ multiset (key/value pairs, duplicates allowed)
 - ✓ Map (one-to-one mapping, out of scope for this course)
 - ✓ Multimap (one-to-many mapping, out of scope for this course)
3. ✓ Derived containers (aka container adapters)
 - ✓ Stack (LIFO)
 - ✓ Queue (FIFO)
 - ✓ priority queue (random insertion/ordered removals)

BASIC SEQUENCE CONTAINERS

- Ordinary C++ array (not part of STL)

- ✓ Fixed size
- ✓ Quick random access (by index number)
- ✓ Slow to insert/delete in the middle
- ✓ Size cannot be changed at runtime

- **vector**

- Relocating, expandable array
- Quick random access (by index number)
- Slow to insert/delete in the middle
- Quick to insert/delete at end



BASIC SEQUENCE CONTAINERS

- list

- ✓ • Doubly linked list
- ✓ • Quick to insert/delete at any location
- ✓ • Quick access to both ends
- • Slow random access

- ✓ • deque

- ✓ • Like vector, but can be accessed at either end
 - ✓ • Quick random access (by index number)
 - ✓ • Slow to insert/delete in the middle
 - Quick insert/delete at either the beginning or the end
-

INSTANTIATING (I.E. CREATING) AN STL CONTAINER

✓ // include appropriate header

//Then use the **template format** with the kind of
// objects to be stored as the parameter

//Examples

→ `vector <int> aVect; //create a vector of ints`

→ `list <airtime> departure_list; //create a list of airtime`
objects

//no need to specify size of containers

COMMON CONTAINER MEMBER FUNCTIONS

- ✓ • **size()**: returns number of element in the container
- ✓ • **empty()**: returns true if empty
- ✓ • **begin()**: returns an iterator to the start of the container, for iterating forwards
- ✓ • **end()**: returns an iterator to the past-the- end location of the container, used to end forward iteration
- ✓ • **rbegin()**: returns a reverse iterator to the end of the container, for iterating backwards
- ✓ • **rend()**: returns an iterator to the beginning of the container, used to end backward iteration

USING STL CONTAINERS

- ✓ • You can create objects of STL containers i.e. list, stack, vector etc.. to use built-in functions of STL containers
- ✓ • You can extend the STL container class by providing your own functions to each class

USING STL CONTAINERS

Example 1

```
#include<iostream>
```

```
#include<list>
```

include library

```
using namespace std;
```

```
void main()
```

```
{
```

```
list<int> l;
```

Declare object

```
l.push_back(5);
```

```
l.push_back(4);
```

```
l.push_back(3);
```

```
l.push_back(2);
```

```
int size=l.size();
```

```
for(int i=0;i<size;i++)
```

```
{
```

```
cout<<l.front();
```

```
l.pop_front();
```

```
}
```

```
}
```

calling Functions



5 4 3 2

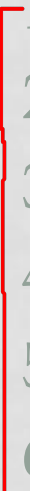
STL ITERATORS

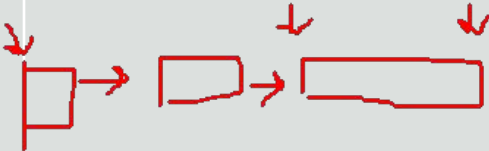
STL COMPONENT 2

STL ITERATORS

- **Similar to pointers**, used to access elements in a container.
- Iterators encapsulate the mechanism used to access container elements.
- This encapsulation enables many of the STL algorithms to be applied to several containers without regard for the underlying container implementation.
- Implemented by iterator classes

ITERATOR CLASSES

- 
1. Forward
 2. Reverse
 3. Bi-directional
 4. Random
 5. Input
 6. Output

Forward Iterator	Reverse Iterator	Bi-directional Iterator	Random Access Iterator
<pre>List<int>::iterator forward = L.begin();</pre>	<pre>List<int>::reverse_iterator backward = L.rbegin();</pre> 	<pre>List<int>::bidirectional_iterator bi = L.rbegin();</pre> OR <pre>List<int>::bidirectional_iterator bi = L.begin();</pre>	<pre>List<int>::random_access_iterator random = L.rbegin();</pre> OR <pre>List<int>::random_access_iterator random = L.begin();</pre>
Can only move forward, one element at a time using ++ operator <pre>forward ++ ;</pre>	Can only move backward, one element at a time using ++ operator <pre>backward ++ ;</pre>	Can move backward and forward, one element at a time using ++, - - operator. For example let iter1 be an iterator, then we can write <pre>iter1 ++ ; to move forward</pre> OR <pre>iter1 - - ; to move backward</pre>	
Cannot be set to an arbitrary value in the middle. i.e. let iter1 be an iterator. We cannot use <u>iter1</u> + n where n is any integer > = 0.			Supports the arithmetic operators + and - between an iterator and an integer value, or subtracting an iterator from another.
Can be compared for equivalence using the equality/inequality operators i.e. let iter1, iter2 be two iterators. They can be compared using == OR != <pre>For Exmple if(iter1 == iter2){ } If(iter1 != iter2) { }</pre>			Can be compared with inequality relational operators (<, >, <= and >=).
Can be dereferenced as an lvalue or rvalue <pre>*forward; OR *forward = 7;</pre>	Can be dereferenced as an lvalue or rvalue <pre>*backward; OR *backward = 14;</pre>	Can be dereferenced as an lvalue or rvalue <pre>*bi ; OR *bi = 24;</pre>	Can be dereferenced as an lvalue or rvalue <pre>*random ; OR *random = 24;</pre>

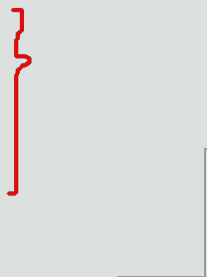
USING FORWARD ITERATOR

Example 2

```
##include<iostream>
#include<list> ←
using namespace std; ←

void main()
{
    list <int> l; ✓
    l.push_back(5);
    l.push_back(4);
    l.push_back(3);
    l.push_back(2);
    int size=l.size();
    list<int>::iterator tempfor;
    for(tempfor=l.begin();tempfor!=l.end();tempfor++)
    { cout<<*tempfor; }
}
```

Declaring an iterator



The diagram consists of a red bracket on the right side of the code, grouping the four `l.push_back()` calls. A red arrow points from the text "Declaring an iterator" to the line `list<int>::iterator tempfor;`. Another red arrow points from the text "Declaring an iterator" to the line `for(tempfor=l.begin();tempfor!=l.end();tempfor++)`.

USING BACKWARD ITERATOR

Example 2

```
##include<iostream>
#include<list>
using namespace std;

void main()
{
    list <int> l;
    l.push_back(5);
    l.push_back(4);
    l.push_back(3);
    l.push_back(2);
    int size=l.size();
    list<int>::reverse_iterator temprev;
    for(temprev=l.rbegin();temprev!=l.rend();temprev++)
        { cout<<*temprev; }
}
```

Declaring an iterator

INPUT AND OUTPUT ITERATORS

- Used for devices
- Input Iterator:
 - Points to an input device (cin or file) to read sequential data items into a container
- Output Iterator:
 - Points to an output device (cout or file) to write data items from the container to the device

STL ALGORITHM

STL COMPONENT 3

STL ALGORITHMS

- An algorithm is a function that does something to items of a container (or containers).
- STL algorithms are implemented as template functions
- In addition to STL container classes, can also work with arrays as well as user-defined container classes

ALGORITHMS

Algorithm	Purpose
find	Returns first element equivalent to a specified value
count	Counts the no. of elements that have a specified value
equal	Compares the contents of two containers and returns true if all corresponding elements are equal
search	Looks for a sequence of values in one container that corresponds with the same sequence in another container
copy	Copies a sequence of values from one container to another (or to a different location in the same container)
swap	Exchange a value in one location with value in another
iter_swap	Exchange a sequence of values in one location with another

ALGORITHMS

Algorithm	Purpose
fill	Copies a value into a sequence of locations
sort	Sorts the values in a container
merge	Combines two sorted ranges of elements to make a larger sorted range
accumulate	Returns the sum of elements in a given range
for_each	Executes a specified function for each element in the container

HOW TO CALL FUNCTIONS IN ALGORITHM

```
✓ #include<algorithm>
using namespace std;
void main( )
{
    int arr[8]= {42, 31, 7, 80, 2, 26, 19, 75};
    → sort(arr, arr+8); // sorts arr
}
```

Diagrammatic annotations in the code:

- A red arrow points down to the number 75 in the array initialization.
- A red arrow points right to the sort function name.
- A red arrow points up to the opening curly brace of the main function.
- A red arrow points up to the opening curly brace of the array initialization.

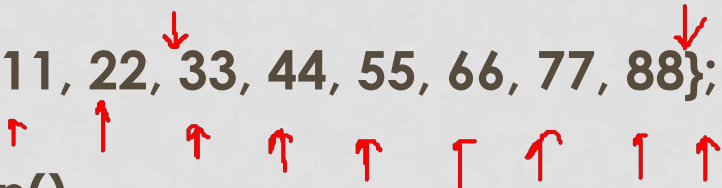
FIND()

- Looks for the first element in a container that has a specified value

FIND() EXAMPLE

```
#include <iostream>
#include <algorithm>
using namespace std;
```

```
int arr[]={11, 22, 33, 44, 55, 66, 77, 88};
```



```
void main()
```

```
{
```

```
    int *ptr;
```

```
    ptr= find(arr, arr+8, 33); // find first occurrence of 33
```

```
    cout<<"First object with value 33 found at offset "
```

```
        <<(ptr-arr)<<endl; //2
```

```
}
```

COUNT()

- Counts how many elements in a container have a specified value and returns this number

COUNT() EXAMPLE

```
#include <iostream>
#include <algorithm>
using namespace std;
```

```
int arr[]={33, 22, 33, 44, 55, 33, 77, 88};
```

```
void main()
{
    int n=✓count(arr, arr+8, 33); //count the number of 33's
    cout<<"There are n<<" 33's in arr. "<<endl;    //3
}
```

SORT()

```
#include <iostream>
#include <algorithm>
using namespace std;

int arr[]={33, 22, 45, -44, 55, 33, 77, 0};

void main()
{
    sort(arr, arr+8);
    for (int j=0; j<8; j++)
        cout<<arr[j]<<' ';
    cout<<endl;
}
```

SEARCH()

- Looks for a sequence of values specified by one container, within another container

SEARCH() EXAMPLE

```
int arr[]={33, 22, 45, -44, 55, 33, 22, 77, 0};  
int pattern[]={33, 22};  
  
void main()  
{  
    int * ptr;  
    ptr=search(arr, arr+9, pattern, pattern+2);  
    if (ptr==arr+9)  
        cout<<"no match found\n";  
    else  
        cout<<"match at "<<(ptr-arr)<<endl; //match at 0  
}
```

Handwritten annotations in red:

- A vertical line above the first '33' in the array.
- A red arrow pointing down to the '55' in the array.
- A red bracket above the '55, 33, 22' in the array.
- A vertical line above the first '33' in the pattern array.
- A red bracket above the '55, 33, 22' in the pattern array.
- A red bracket above the '55, 33, 22' in the pattern array, with the text {55, 33, 22} written in red next to it.
- A vertical line above the '9' in 'arr+9'.
- A vertical line above the '2' in 'pattern+2'.
- A vertical line below the '0' in the output string.

MERGE()

- Works with three containers, merging the elements from two source containers into a destination container

MERGE() EXAMPLE

```
int src1[]={2, 3, 4, 6, 8};  
int src2[]={1, 5, 7};  
int dest[8];
```

```
void main()  
{  
    merge(src1, src1+5, src2, src2+3, dest);  
    for (int j=0; j<8; j++)  
        cout<<dest[j]<<' ';    //1 2 3 4 5 6 7 8  
    cout<<endl;  
}
```

FOR_EACH()

- Allows you to do something (a function) to every item in a container
- Your function cannot change the elements in a container, but it can use or display their values

FOR_EACH()

```
void in_to_cm(double in)
{
    cout<<(in*2.54);
}

void main()
{
    double inches[]={3.2, 6.5, 1.0, 12.56, 4.33};
    for_each(inches, inches+5, in_to_cm);
}
```

Diagram illustrating the arguments passed to the `for_each` function:

- `inches` (array) is passed as the first argument.
- `inches+5` (end iterator) is passed as the second argument.
- `in_to_cm` (function) is passed as the third argument.

TRANSFORM()

- Does something (a function) to every item in a container, and places the resulting values in a different container (or the same one)

TRANSFORM()

```
double in_to_cm(double in)
{
    return (in*2.54);
}

void main()
{
    double inches[]={3.2, 6.5, 1.0, 12.56, 4.33};
    double centi[5];
    transform(inches, inches+5, centi, in_to_cm);
    for (int j=0; j<5; j++)
        cout<<centi[j];
}
```

READING ASSIGNMENT

- Read Appendix F from Object Oriented Programming C++ by Robert Lafore, 4th Edition, Chapter 15

BOOK READING

- • Chapter 15, Object Oriented Programming C++ by Robert Lafore, 4th Edition,

REFERENCES

- www.cplusplus.com